AI Gateway — Architecture & Technique Analysis

3-phase routing architecture · evidence-labeled technique comparison · feature-flag ladder · assumption register

ARVAN CLOUD AI GATEWAY · SHARIF UNIVERSITY · DESIGN PHASE · GENERATED 2026-07-24

AI Gateway — Final Architecture

Merged design: pragmatic spine (Design A) + staged research ceiling (Design B) Arvan Cloud / Sharif University — design phase, v1.1

0. Design stance, stage map, and notation

Stance. The hot path is boring by construction: config rules, hash lookups, ledger arithmetic, and at most one 1.5B router model. Everything trained, judged, or clever runs offline or async until logged traffic proves it pays. Design A's MVP pipeline is the permanent spine; every Design B advanced component is retained as an **independently disable-able feature flag** with a measurable activation condition and a measurable kill condition (§6). Nothing in a later stage changes the Phase-1/2/3 shape — later stages only flip flags inside it.

Build stages (mapped to the strategy brief's roadmap):

Stage	Roadmap phase	Contents in one line
MVP	Phase 1	Rules + Arch-Router routing, single-shot dispatch, budget ledger + watchdog, exact cache, tier-0 verifiers, PII redaction + input/output guards, template XAI, promptfoo + dry-run, quantized tiers manufactured offline
Stage-2	Phase 2	Sequential cascade, semantic cache (raced, vCache), logprob deferral, length predictor, hedging, deadline→priority mapping, history summarization (gated), reasoning-tier thinking-budget dispatch, output-guard classifier layer
Stage-3	Phases 3–4	Best-of-N, refine loop, speculative parallel cascade, RM scorer, neural compression (gated + hardened), trained routers / learned quality head / few-shot onboarding proper, Krites, MoA (flag, default OFF)

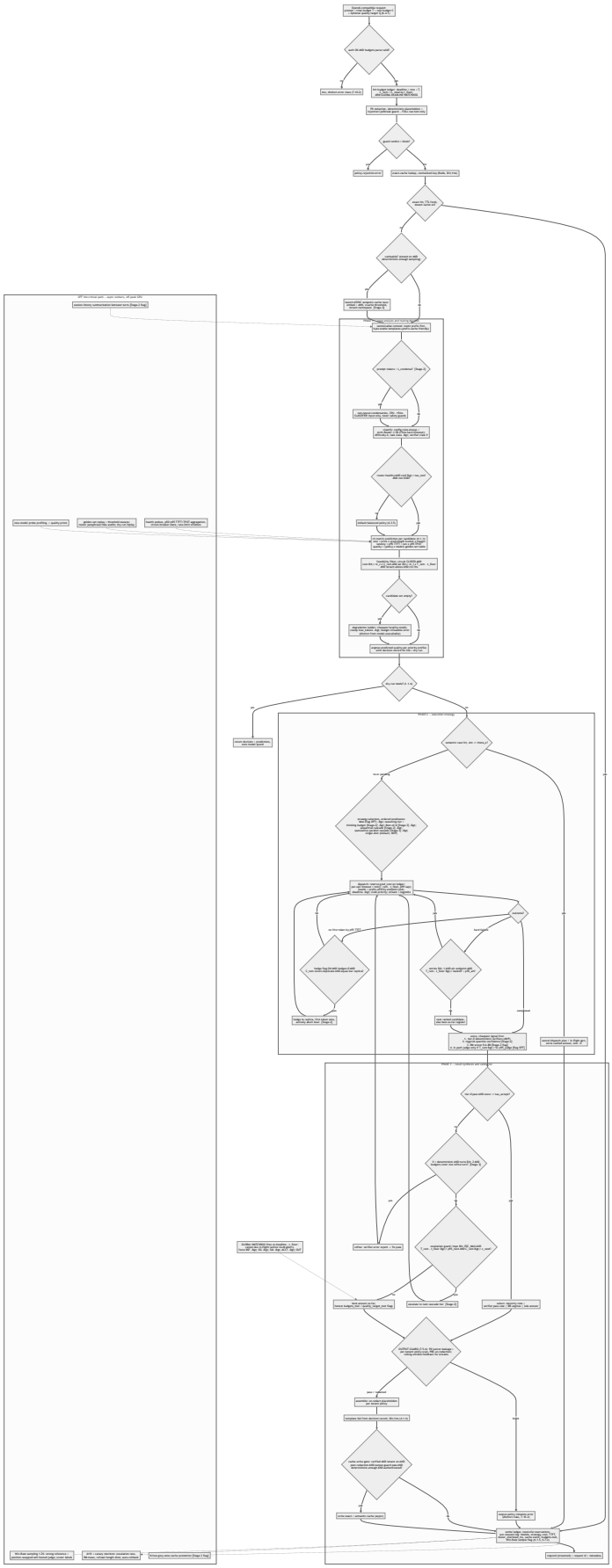
Named tunables (YAML, per-tenant overridable, GitOps-versioned per reqs 1.2.3–1.2.5; every value calibrated by RouterBench-style golden-set replay in dry-run mode and re-swept on each model/policy change):

Symbol	Meaning	Default	Calibrated by
`T_rem`, `C_rem`	Remaining time/cost budget (ledger)	per request	—
`t_floor`	Reserved tail for Phase-3 scoring + assembly (`SYNTH_RESERVE`)	max(200ms, 2%·T)	measured Phase-3 p95
`m_t`, `m_c`	Safety margins on latency/cost predictions	0.85 / 0.90	length-predictor error (±20–40%)
`ROUTER_TIMEOUT`	Arch-Router hard timeout	75ms	router p95
`τ_conf`	Router confidence floor	0.6	paraphrase-consistency audit
`d_hard`, `[d_lo, d_hard)`	Hard threshold / ambiguous band of difficulty `d`	0.7 / tuned	golden-set sweep
`V`	Verifier class ∈ {deterministic, vote, rm, judge, none}	per task class	task taxonomy
`θ_e`, `θ_lo`	Semantic-cache per-entry threshold (vCache, error ε ≤ 1%) / Krites grey-zone floor	learned	vCache online / replay
`L_condense`	Router-input condensation trigger	2048 tokens	prompt-length telemetry
`K_think`	Max thinking-token cap for the reasoning tier (S2T); effective cap derived per request from `T_rem` / `C_rem`	per policy	golden-set uplift-vs- `K` curve (§5.6 probes)

Symbol	Meaning	Default	Calibrated by
`N_MAX`, `ESC_MAX` / `max_hops`, `REFINE_MAX_TURNS`, `HEDGE_MAX`	Strategy caps	5 / 1 / 2 / 1	golden-set uplift curves
`τ_lp`, `τ_rm`, `τ_accept`	Logprob-quantile / RM / composite acceptance thresholds	per (model, policy)	golden-set sweep; recalibrate on every model swap (release gate)
`JUDGE_SAMPLE`	Async Win-Rate sampling rate	1–2%	budget
`ĉ(m)`, `p95_lat(m)`, `p95_TTFT(m)`	Predicted cost / observed p95 latency / TTFT per endpoint	live telemetry	Prometheus (2.3.1, 6.2.3)

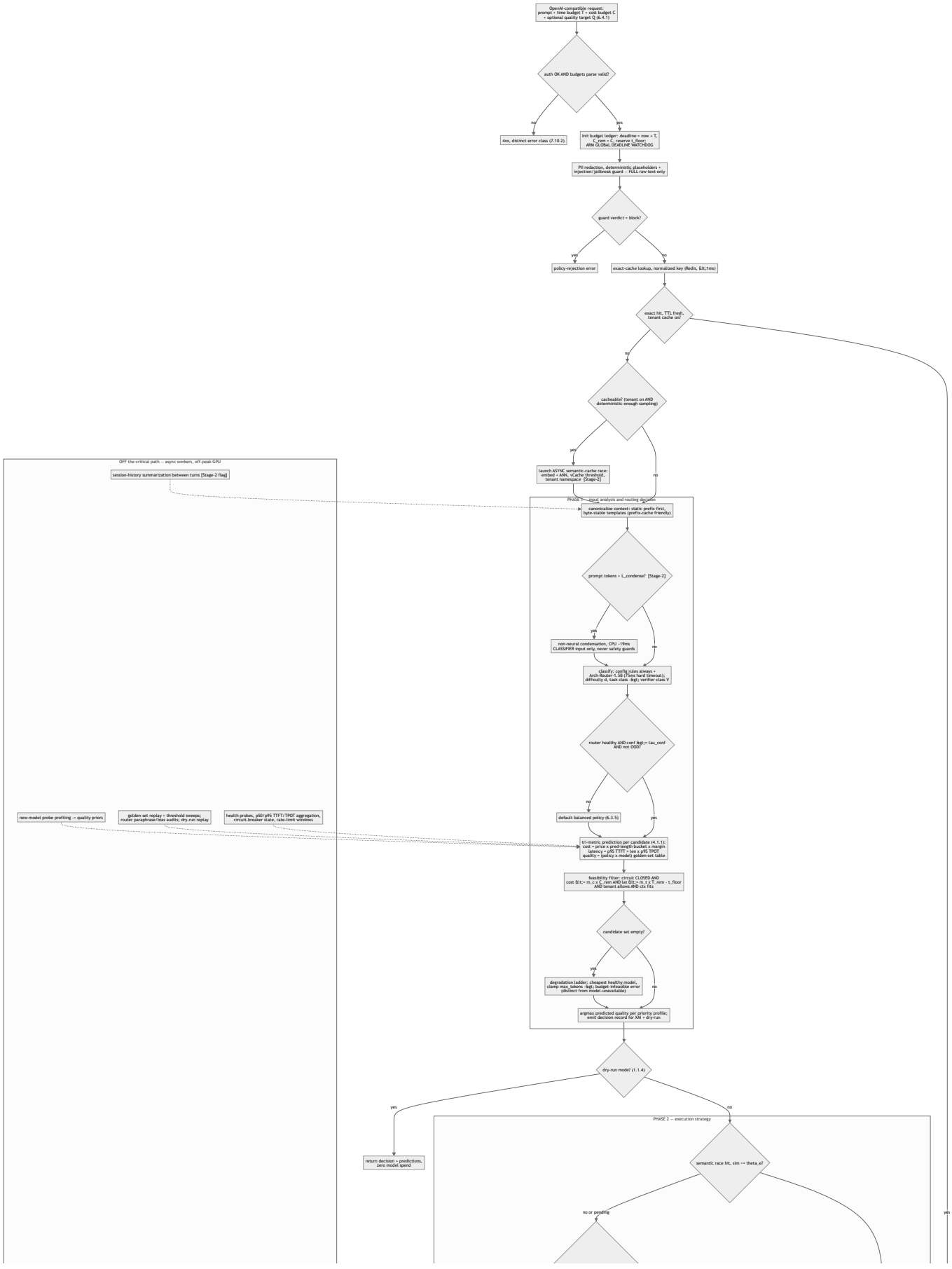
1. End-to-end flow

End-to-end request flow — overview (fit to page)

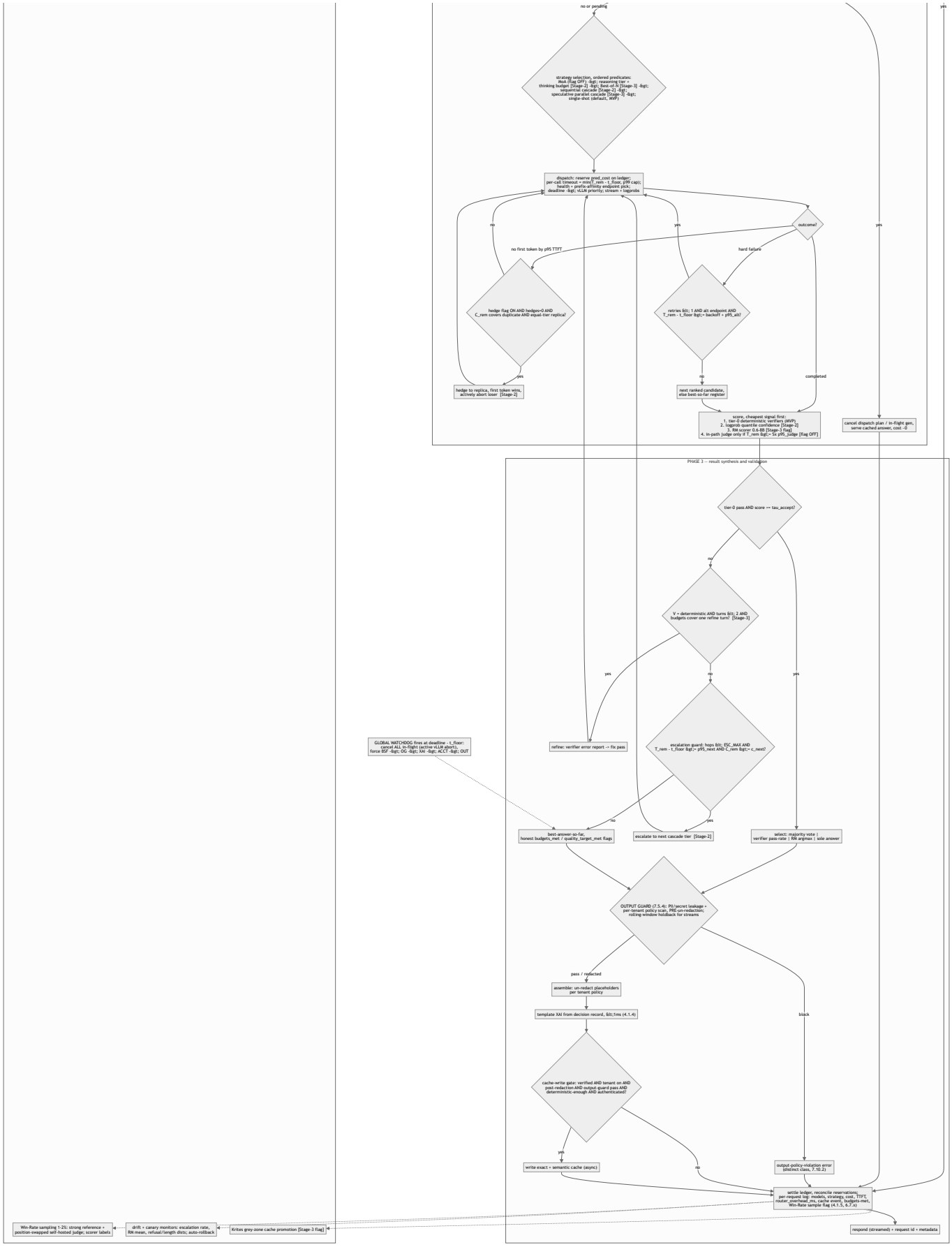


Readable full-width slices follow on the next pages. The mermaid source lives in Docs/AIG-Architecture-Analysis.md.

Request flow — detail 1/2



Request flow — detail 2/2



The **global deadline watchdog** (armed at ledger init, fires at `deadline − t_floor` regardless of pipeline position) is what makes "never knowingly exceed the time budget" (6.4.3, 7.1.1) structural rather than aspirational: p100 latency is bounded by construction, and the only permitted overshoot is the unavoidable cost of returning the best-so-far answer itself.

---

2. Phase 1 — Input analysis & routing decision

Inputs: OpenAI-compatible request; `T`, `C`, optional quality target `Q`; tenant id + priority profile (max-quality / min-cost / min-latency, 4.1.2); session id (1.1.3); model-registry view (id, modality, ctx limit, price/token, quality priors, live p50/p95 TTFT/TPOT, health/circuit state — 6.2.2/6.2.3); dry-run flag; initialized ledger.

Outputs: Decision record — winning policy name, difficulty `d`, task class → verifier class `V`, ranked feasible candidates each with predicted (quality, cost, latency), designated cheap/strong tiers, strategy hints, XAI feature set, dry-run counterfactual entry; redacted canonicalized prompt.

Ordered steps

#	Step	Mechanism	Hot-path cost	Stage
1.1	Constraint interpretation: parse `T` / `C` / `Q`, merge tenant priority profile, init ledger, **arm watchdog**	budget ledger (§5.1)	<0.1ms	MVP
1.2	PII redaction + injection/jailbreak guard on **full raw text**, deterministic session-stable placeholders (`<PII:EMAIL:1>`)	regex+dictionary MVP; NER model Stage-3	1–5ms	MVP
1.3	Exact-cache lookup, normalized key `hash(tenant, policy, redacted_prompt, sampling params)`	Redis tier-0	<1ms	MVP
1.4	Fire semantic-cache race (non-blocking): co-located embed + ANN, vCache per-entry threshold, tenant namespace	raced, 0ms added on miss	0ms	Stage-2
1.5	Context canonicalization: static system/policy blocks first, volatile user content last, byte-stable (prefix-cache discipline)	free	~0ms	MVP
1.6	Long prompts only: non-neural condensation (TextRank/TF-IDF, CPU) of the **classifier input only**; full prompt still goes to the target model; **never** to safety classifiers	`router.condense` flag	≤20ms, only > `L_condense`	Stage-2 (gated)
1.7	Classification: (a) config-rule/CEL tiering — always-on permanent floor, <1ms; (b) Arch-Router-1.5B intent→policy on GPU slice, hard 75ms timeout → rules fallback. Output: policy, `d`, task class → `V`, votable/code/RAG/multi-turn flags	`router.rules` core, `router.arch` flag	<1ms / 51±12ms	MVP
1.8	Tri-metric prediction (4.1.1): **cost** = price/token × bucketed output-length prediction × margin; **latency** = p95 TTFT + len × p95 TPOT (live telemetry); **quality** = static (policy × model) golden-set table. Length: heuristic per-task priors MVP → DistilBERT ~4ms Stage-2 → learned quality-gap head Stage-3	Hybrid-LLM substrate	<1ms (+4ms Stage-2)	MVP / Stage-2 / Stage-3
1.9	Feasibility filter + rank: drop infeasible (predicate D5); argmax predicted quality; min-cost/min-latency profiles invert sort inside a quality floor	core	<0.1ms	MVP
1.10	Robustness rails: OOD/low-confidence → default policy; gadget-suffix sanitization; per-tenant expensive-tier cost-anomaly counters	`router.harden`	<1ms	MVP
1.11	Emit decision record + XAI features + dry-run counterfactual; branch to dry-run return if flagged	core	<0.1ms	MVP

Decision predicates

ID	Predicate	True	False
D0	`auth_ok ∧ T > 0 ∧ C > 0 ∧ well_formed`	proceed	4xx, distinct error class (7.10.2)

ID	Predicate	True	False
DG	`guard_verdict = block` (full-text injection score > τ_inj or tenant policy)	policy-rejection error	proceed redacted
D2	`exact_key ∈ Redis ∧ age < TTL(policy) ∧ tenant_cache_on`	serve, debit ≈ 0, jump to accounting	continue
D1	`tenant.cache_enabled ∧ ¬opt_out ∧ sampling deterministic-enough`	launch semantic race (never blocking)	skip
D3	`prompt_tokens > L_condense`	condense classifier input only	classify full text
D4	`router_lm.healthy ∧ latency ≤ ROUTER_TIMEOUT ∧ conf ≥ τ_conf ∧ ¬OOD`	use router policy	rules → default balanced policy (6.3.5)
D5	`∃m: ĉ(m) ≤ m_c·C_rem ∧ pred_lat(m) ≤ m_t·T_rem − t_floor ∧ circuit(m)=CLOSED ∧ tenant_allows(m) ∧ ctx_fits(m)`	rank candidates	degradation ladder: cheapest healthy model with clamped `max_tokens` → `budget-infeasible` error, distinct from `model-unavailable` (7.10.2)
DD RY	`dry_run = true`	return decision + predictions, zero spend (1.1.4)	Phase 2

Failure / fallback paths

- Arch-Router down/slow → rules-only (6.3.5); rules are the permanent floor, never removed. Auto-degrade if `router_overhead_ms` p95 > 2× target for 15 min.
- PII classifier down → fail-closed per tenant policy: block off-boundary dispatch or restrict to the designated in-boundary local model.
- Length predictor bucket accuracy < 60% rolling → auto-fallback to static p90 priors, flag off.
- Registry stale → last-known-good ConfigMap (2.2.1 hot reload); Redis down → cache flags auto-off via circuit breaker, pipeline continues uncached.
- All models unhealthy → `model-unavailable` error, or stale-within-TTL cache entry if tenant policy allows.

Off the critical path

Health probes + latency aggregation (feeds 1.8); golden-set profiling filling the quality table; paraphrase-consistency and category-bias router audits (dry-run replay); threshold sweeps; length-predictor training on logged traffic; Arch-Router policy authoring.

---

3. Phase 2 — Execution strategy

Inputs: decision record (ordered candidates + escalation tier, `d`, `V`, tags), ledger (`T_rem`, `C_rem`), semantic-race handle, streaming flag, per-endpoint health/queue/hedge telemetry, effective flags.

Outputs: one or more raw candidate answers + logprobs + per-call actuals settled into the ledger; best-so-far register updates; execution trace for Phase 3 and XAI.

Strategy selection — ordered evaluation, first match wins (pure functions of decision record + ledger + telemetry, <1ms total)

ID	Strategy	Predicate	Flag	Stage
S0	**Serve semantic hit**	race resolved ∧ `sim ≥ θ_e` (vCache, error ≤ ε) ∧ TTL fresh — cancels dispatch plan or in-flight generation within first tokens	`cache.semantic`	Stage -2

ID	Strategy	Predicate	Flag	Stage
S1	**MoA ensemble**	`Q = max_quality ∧ C_rem ≥ 5·ĉ(single) ∧ T_rem ≥ 2.5·p95(single) ∧ V = none` (open-ended, no cheap verifier — otherwise S2 dominates)	`exec.moa`, **default OFF**	Stage-3+
S2T	**Reasoning-tier single-shot (thinking-budget knob)**	`d ≥ d_hard ∧ reasoning_tier ≠ ∅ ∧ T_rem ≥ p95_reason + t_floor ∧ C_rem ≥ ĉ_ans + ĉ_think(K)`; `K = min(K_think, ⌊(m_c·C_rem − ĉ_ans)/price_tok⌋, ⌊(m_t·T_rem − t_floor)/TPOT − pred_len_ans⌋)` — the remaining T/C budget maps to a **max-thinking-tokens cap**, reserved and debited on the ledger like any generation. Evaluated **before** S2 and before refine: the reasoning model's built-in knob is the premium dial, preferred over hand-built orchestration (strategy brief, Phase-3 roadmap — "prefer a reasoning model's built-in knob over building orchestration")	`exec.think budget`	Stage-2
S2	**Best-of-N parallel**	`d ≥ d_hard ∧ V ∈ {vote, deterministic} ∧ C_rem ≥ N·ĉ(m) + ĉ_score ∧ T_rem ≥ p95_batch(m,N) + t_floor ∧ gpu_util < 0.7`; `N = min(N_MAX, ⌊C_rem/ĉ(m)⌋)`, vLLM `n>1` shares prefill; reached only when S2T is off, infeasible, or beaten per dollar-second on this slice (§6, §8)	`exec.bon`	Stage-3
S3	**Sequential cascade**	`d ∈ [d_lo, d_hard)` (ambiguous) ∧ cheap tier feasible ∧ `T_rem ≥ p95(cheap) + p95(strong) + t_floor`; cheap tier runs abstention-prompted	`exec.cascade`	Stage-2
S4	**Speculative parallel cascade**	cascade wanted ∧ time predicate fails ∧ `T_rem ≥ p95(strong) + t_floor ∧ C_rem ≥ ĉ(cheap) + ĉ(strong)` — fire both, actively abort strong on cheap-accept	`exec.speccascade`	Stage-3
S5	**Single-shot routed**	default: everything else (`d` low, or budgets too tight for anything richer). Client streaming allowed here only at MVP	core	MVP
—	Refine loop	never selected up front — Phase-3 trigger only (DREF)	`exec.refine`	Stage-3
—	Multi-model debate	**EXCLUDED** — dominated by self-consistency at equal compute (Smit et al.)	—	never

S2T targets self-hosted reasoning models (Qwen3-class thinking mode / DeepSeek-R1 distills served on vLLM); the cascade's strong tier and the reasoning tier may share hardware (A15). The watchdog may truncate thinking mid-stream — the answer segment is still produced from whatever thinking completed, so budget violations remain structurally impossible.

Execution mechanics (all strategies)

#	Step	Rule	Stage
2.1	Reserve	Debit ledger with `pred_cost` **before** dispatch; settle to actuals on completion; refund on cancellation. Parallel strategies reserve the full batch up front — the cap is enforced pre-spend. S2T reserves `ĉ_ans + ĉ_think(K)` — thinking tokens are ledger debits like any other tokens	MVP
2.2	Per-call timeout	`min(T_rem − t_floor, p99_cap(endpoint))` on every downstream call (6.5.3); expiry cancels the vLLM request server-side	MVP
2.3	Endpoint pick	Health/least-load MVP; + session/tenant consistent-hash prefix-affinity across replicas (`serve.prefixlb`, payoff at >1 replica) Stage-2; remaining-deadline → vLLM `--scheduling-policy priority` (`serve.priority`) Stage-2; tight-`T` requests → speculative-decoding latency-tier endpoints (`serve.specdecode`, serving-layer config) Stage-2	
2.4	Hedging	No first token by p95 TTFT ∧ `hedges = 0` ∧ `C_rem ≥ duplicate` ∧ equal-tier replica → duplicate, first-token-wins, **actively abort** loser; +2–5% cost cap	Stage-2 (`exec.hedge`)
2.5	Retry	Hard failure → max 1 retry, alternate endpoint, exponential backoff + jitter, only if `T_rem − t_floor ≥ backoff + p95_alt`; per-endpoint circuit breakers — open circuit = skip instantly to local fallback (7.4.2, 6.5.5); retries debit the ledger	MVP

#	Step	Rule	Stage
2.6	Cancellation discipline	Every parallel pattern (spec-cascade, BoN, hedges, semantic-race losers, watchdog) must actively abort losing vLLM requests to reclaim GPU — otherwise the ledger says "saved" while the fleet pays double	MVP (built with first parallel feature)
2.7	Streaming + capture	Tokens stream to Phase-3 scorer (and client if `stream=true`, via the output guard's rolling-window holdback — §4 step 3.9); logprobs captured free on vLLM; best-so-far register updated on every completed candidate. Closed-API endpoints may lack logprobs → scorer chain degrades (§4)	MVP
2.8	Thinking-budget mapping (S2T)	Set the serving-layer max-thinking-tokens sampling param to `K`; watchdog truncation forces transition to the answer segment; the uplift-vs-`K` curve per policy comes from §5.6 probe profiling and drives `K_think`	Stage-2

Failure / fallback paths

- Call fails, retry exhausted → next ranked Phase-1 candidate (7.4.1); candidates exhausted → Phase 3 with best-so-far (possibly nothing → structured error).
- Cheap tier **abstains** → immediate escalation, skipping full generation + scoring (early-abstention shortcut, prompted variant — free).
- Strong tier unreachable during escalation → return accepted-or-best cheap answer, `escalation_failed=true`.
- Reasoning tier saturated or unhealthy → S2T predicate fails closed; selection falls through to S2/S3/S5 exactly as if the flag were off.
- GPU util ≥ 0.85 sustained → BoN and speculative/hedge predicates soft-disable themselves; single-shot + sequential cascade only.
- Closed-API tier vanishes (sanctions) → circuit OPEN within k failures; all policies re-anchor on self-hosted tiers; no user-visible outage (7.4.4).
- Ledger hits zero or watchdog fires → cancel all in-flight, jump to Phase-3 best-so-far immediately.

Off the critical path

Loser/hedge cancellation reaping; provider rate-limit window tracking; KubeAI autoscaling / scale-to-zero signals; quantized-variant fleet production (AWQ-INT4/FP8) + golden-set re-validation — this offline work **manufactures the cheap tier** the router monetizes; queue/priority telemetry deciding whether anything beyond priority mapping is ever warranted.

---

4. Phase 3 — Result synthesis & validation

Inputs: candidate answer(s) + logprobs + verifier artifacts, execution trace, decision record, ledger, best-so-far register.

Outputs: final answer; machine-readable metadata (models used, strategy, cost, latency, budgets-met flags, request id — 6.7.1–6.7.3); XAI explanation; settled ledger; per-request log + metrics; cache writes; async labeling jobs.

Ordered steps

#	Step	Mechanism	Cost	Stage
3.1	Tier-0 deterministic verification: JSON schema, regex/format/length sanity; sandboxed code exec with hard timeout, code slice only	mandatory first scorer layer	0–5ms (code exec 100ms–2s bounded)	MVP

#	Step	Mechanism	Cost	Stage
3.2	Logprob **quantile** confidence (not raw mean; per-model calibrated `τ_lp` — recalibration is a release gate on every model swap)	free from vLLM	~0ms	Stage-2 (`score.logprob`)
3.3	Reward-model score (Skywork-Reward-V2 0.6–8B class, single prefill pass) for open-ended answers; also the BoN selector for `V=rm`	co-located GPU	10–50ms	Stage-3 (`score.rm`)
3.4	In-path LLM judge — only `Q set ∧ V=none ∧ T_rem ≥ 5·p95_judge ∧ C_rem ≥ ĉ_judge` (high-budget slice)	self-hosted 7–14B, pairwise	300ms–3s, gated	Stage-3 (`score.judge_inpath`, default OFF)
3.5	Acceptance decision: composite of 3.1–3.4 vs `τ_accept(policy, budget)`	golden-set-swept thresholds	<0.1ms	MVP (tier-0 only) → Stage-2
3.6	Multi-candidate selection: votable → majority vote; verifier-available → pass-rate then confidence; else RM argmax / highest confidence (6.5.2)	—	<1ms	Stage-3 (with BoN)
3.7	Refine trigger — see DREF; **never refine without a verifier**	verifier error report fed back, same model	+1 gen/turn	Stage-3 (`exec.refine`)
3.8	Escalation — see DESC; deadline-aware guard	cascade escalation	—	Stage-2
3.9	**Output guard (7.5.4)**: scan of the selected / best-so-far answer **before un-redaction and before client delivery** — (a) regex + dictionary layer for raw PII patterns (any raw PII in generated text is leakage by construction, since the input was redacted to placeholders), secrets/credentials (key/token formats, high-entropy strings), and per-tenant policy terms; (b) the same co-located classifier class used on the input side (step 1.2) for sensitive-data leakage and policy-violating/harmful content, per-tenant configurable. Because un-redaction happens only after this gate and reinserts only the requester's own placeholder map, the delivered answer cannot introduce third-party leakage the guard did not see. **Streaming interaction**: streamed responses pass through a rolling-window holdback buffer — tokens are released to the client only after the trailing window clears the regex layer; a hard match mid-stream terminates the stream with a policy-violation event and the partial text is quarantined in the audit log, not delivered	`guard.output` — failure path in DOG below	1–5ms regex; +5–15ms classifier	MVP (regex layer) / Stage-2 (classifier layer)
3.10	Assembly: final answer + metadata; un-redact placeholders from per-request map only if tenant policy allows **and 3.9 passed**; `quality_target_met:false` when Q unmet within budget (6.4.8)	—	<0.1ms	MVP
3.11	XAI render: template over decision-record fields (§5.5)	string formatting	<1ms	MVP
3.12	Cache-write gate + write (exact MVP; + semantic Stage-2): verified answers only, post-redaction, output-guard-passed (DCW consumes the 3.9 verdict for `¬pii_residue`), tenant-namespaced, per-policy TTL; async, never blocks response	poisoning defense (§5.4)	<1ms	MVP / Stage-2
3.13	Accounting close: reconcile reservations vs actuals; per-request structured log — models, strategy, tokens, cost, TTFT, total latency, `router_overhead_ms`, cache event, output-guard verdict, Win-Rate sample flag, Cost Savings vs always-strong counterfactual, budgets-met (4.1.5, 7.3.2, 7.6.1)	Prometheus + log flush async	<1ms inline	MVP

Decision predicates

ID	Predicate	True	False
DACC	`tier0_pass ∧ composite_score ≥ τ_accept(policy, budget)`	accept → select	→ DREF
DREF	`V = deterministic ∧ verifier_failed ∧ turns < REFINE_MAX_TURNS ∧ T_rem − t_floor ≥ p95_turn ∧ C_rem ≥ ĉ_turn`	one refine turn → Phase-2 dispatch	→ DESC
DESC	`hops < ESC_MAX ∧ next_tier ≠ ∅ ∧ T_rem − t_floor ≥ p95_lat(next) ∧ C_rem ≥ ĉ(next)` (also fires on cheap-tier **abstention**, skipping scoring)	escalate → Phase-2 dispatch	best-so-far, honest `budgets_met` flags — never a knowing overrun (6.4.3)
DOG	`guard_out ∈ {pass, redacted-in-place}` — pattern-local hits (a stray email, a key-shaped string) are redacted in place and the response is flagged `output_guard=redacted`; anything beyond local redaction is a block	assemble + un-redact (3.10)	`output-policy-violation` error — a **distinct error class** (7.10.2), never conflated with `budget-exhausted` or `model-unavailable`; ledger still settles, full audit log entry written
DQ	`Q set ∧ best_score < Q`	best answer + `quality_target_met=false` (6.4.8)	return normally
DCW	`verified ∧ tenant_cache_on ∧ ¬pii_residue (from 3.9) ∧ temperature ≤ τ_cachetemp ∧ authenticated_generation`	write caches	skip

Failure / fallback paths

- Scorer component down → degrade the chain in place: tier-0 → logprob → accept-with-flag; never block a response on a scorer.
- Output-guard classifier down → degrade to the regex/dictionary layer only, flag `output_guard=degraded`; the regex layer is never skipped — it is a mandatory floor like `router.rules`.
- Sandbox hang → hard timeout; treat as verifier-fail, not request-fail; fall through on confidence.
- All candidates failed, no budget left → best-so-far by (verifier partial score, RM, logprob) ordering; nothing at all → structured error distinguishing `budget-exhausted` / `model-unavailable` / `invalid-input` / `output-policy-violation` (7.10.2).
- Cache write failure → log and drop.
- Watchdog during scoring → skip remaining scorers, but **never** the output guard: assemble best-so-far through 3.9 → 3.10 immediately (the guard's cost is part of `t_floor`).

Off the critical path

Semantic-cache writes; Win-Rate judge sampling (1–2%, position-swapped, pinned strong reference) → 4.1.5 metric + labels bootstrapping the Stage-3 RM/DistilBERT scorer; Krites grey-zone verification → cache promotion (Stage-3 flag); drift alarms (escalation rate, RM mean, refusal/length distributions, output-guard hit rate); judge-scored canary windows on blue-green rollouts with auto-rollback; end-user feedback ingestion keyed on request id (6.7.2); session summarization between turns.

---

5. Cross-cutting mechanics

5.1 Budget ledger + watchdog (the spine)

One in-memory object per request in request context only — stateless-core compatible (5.1); session state lives in Redis.

Field / rule	Semantics
`request_id, tenant, policy, flags_effective`	correlation (6.7.2) + reproducibility snapshot of active flags (feeds XAI)

Field / rule	Semantics
`deadline = arrival + T` (monotonic clock); `t_floor` reserved	`T_rem = deadline − now`; watchdog armed at init, fires at `deadline − t_floor`
`C_total, C_reserved, C_spent, debits[]`	reservation debited **before** every dispatch/retry/hedge/sample/refine/scorer call — thinking tokens included — using predicted cost; reconciled to actuals; refunds on cancellation. One ledger, no side channels — a cap that hedges don't debit is fiction (7.3.1)
`best_so_far`	{answer, composite score, source model, verified?} — updated per completed candidate; what the watchdog returns (6.4.3)
`hops, turns, retries, hedges`	strategy caps
Exhaustion	`T_rem ≤ t_floor` or `C_rem ≤ 0` at any checkpoint → cancel in-flight (active vLLM abort reclaims GPU), best-so-far path
Outcomes	`budget-exhausted-with-answer` ≠ `budget-infeasible-no-answer` ≠ `model-unavailable` ≠ `output-policy-violation`: four distinct client-visible outcomes (7.10.2)
Cache hits	debit ≈ 0 marginal cost (6.6.2), surfaced in metadata + XAI; accounting hook built at **MVP, before any semantic cache exists**, so dry-run and promptfoo exercise it from day one

5.2 Hot-path allowlist (4.2.1 / 6.3.6)

Allowed inline	Budget	Banned inline (async/offline only)
CEL/rule eval, ledger arithmetic, feasibility filter, rank	<1ms	LLM-judge scoring (except gated 3.4 slice)
Exact-cache Redis lookup	<1–5ms	Blocking semantic-cache lookup (raced instead)
PII/guard classifiers (co-located)	1–5ms	Per-request LLM-generated explanations
Output guard: regex layer (+classifier Stage-2)	1–5ms (+5–15ms)	Neural prompt compression (until Stage-3 gate)
Length predictor (Stage-2)	~4ms	Reference generation for Win Rate
Non-neural condensation (only > `L_condense`)	≤20ms	Golden-set replay, threshold sweeps, training
Arch-Router-1.5B (hard timeout)	51±12ms — the ceiling	Synchronous log flushes, Krites judging
Semantic embed + ANN	5–20ms **raced** → 0ms on miss	
Logprob check, template XAI, accounting	~0–1ms	

Targets, logged per request as `router_overhead_ms`: **p95 ≤ 10ms rules-only path; ≤ 75ms full ML path** — both negligible vs seconds-scale generation. Sustained 2× breach for 15 min auto-degrades `router.arch` to rules-only.

5.3 PII redaction & I/O guard placement (1.1.2, 4.2.4, 3.2.2, 7.5.4)

Input side. Immediately after authN, **before** any cache-key computation, any embedding, any classifier other than the guard itself, any logging, and any dispatch. Injection/jailbreak guards run on **full text only** (condensation degrades jailbreak detection −14.2pp — condensed text is for the intent classifier alone). Deterministic session-stable placeholders keep prompts byte-stable (prefix caching survives) and make cache keys/entries PII-free by construction; placeholder map lives in per-request/session Redis context, never pod memory or logs. Un-redaction once, at assembly, tenant-permitting. Per-tenant policy may forbid dispatch of even-redacted content off-infrastructure (7.9.1). Regex+dictionary at MVP; NER-grade Stage-3, gated on audit-log misses.

Output side (7.5.4 mandates both directions). Every generated answer — accepted, best-so-far, or watchdog-forced — passes the output guard (P3 step 3.9) **before** un-redaction, cache-write, and client delivery. The guard runs pre-un-redaction deliberately: at that point any raw PII, secret, or credential in the generated text is leakage by construction (the model never saw the originals from this request), so detection is high-precision; un-redaction afterwards reinserts only the requester's own data and cannot add third-party leakage. Per-tenant policy configures the term dictionaries, classifier categories, and the redact-vs-block preference (DOG). Streamed responses are guarded through a rolling-window holdback buffer (step 3.9); the buffer depth is a latency/coverage tunable counted inside `t_floor`. The regex layer ships at

MVP and is a mandatory floor (like `router.rules`); the classifier layer reuses the co-located input-guard model class at Stage-2 and auto-degrades to regex-only on latency or health breach — it never silently disappears (`guard.output`, §6).

5.4 Cache architecture & accounting (5.2, 6.6.x)

- **Tier-0 exact** (MVP): normalized-key Redis, zero false-positive risk, blocking <1ms; the only tier safe for fuzzy-match opt-out tenants (6.6.4).
- **Semantic** (Stage-2): **raced**, never blocking — the only design compatible with both 5.2 and 4.2.1. Co-located MiniLM-class embedder + ANN; **vCache per-entry learned thresholds with error bound ε ≤ 1% is the default policy, not an upgrade** (the optional per-request quality target maps onto ε). A hit resolving after generation started cancels within first tokens (~1–5% wasted prefill on those hits).
- **Isolation & poisoning defense** (ships with Stage-2, designed-in at MVP): per-tenant namespaces, never shared across paying tenants (CacheAttack: 80–88% hijack success against production AWS/Azure shared caches); verified-write gate — only authenticated, post-redaction, output-guard-passed, quality-accepted generations are written.
- **Governance**: per-tenant/per-policy TTL + invalidation (3.1.3, 6.6.1); hits logged as near-zero marginal cost (6.6.2) and `served_from=cache` in XAI/metadata. Re-baseline the ">30% hit rate" Stage-2 gate on logged Iranian-market traffic — the 31% literature anchor is a 20-user study (tagged [evidence→assumption] in §8).
- **Krites grey-zone promotion** (Stage-3 flag): `θ_lo ≤ sim < θ_e` → async rubric-judge verification → curated-tier promotion; zero critical-path latency; enable only if FAQ-heavy traffic materializes and self-hosted-judge agreement ≥ 90% on golden pairs.

5.5 XAI (4.1.4)

Template-first, **faithful by construction**: rendered <1ms from the decision record the router actually used — policy name (Arch-Router's output doubles as the human-readable label), difficulty band, per-candidate predicted quality/cost/latency, the binding constraint ("Model B excluded: predicted latency 8.2s > remaining budget 5s"), strategy chosen and why, cache/cascade/escalation/hedge/fallback events, budget outcomes. Free-form per-request LLM rationales are rejected (documented unfaithfulness + latency). Optional Stage-3 polish: LLM-phrased wording generated once per (policy × model × budget-band) cell offline, cached, amortized ≈ 0.

5.6 Zero-downtime model onboarding (4.1.3, 5.5, 6.2.4)

1. **Register** via YAML through GitOps (id, endpoint, modality, ctx limit, price basis, capability tags); syntax-validated at load (2.2.2); hot reload, no pod restart (2.2.1), no client change (6.8.2); modular adapter (7.7.2).
2. **Probe-profile offline** (minutes–hours of batch inference on owned H100s): golden set + fixed UniRoute-style probe set → per-policy quality prior vector, TTFT/TPOT p50/p95, output-length calibration, per-model `τ_lp` scale; reasoning-tier models are additionally probed at 3–4 thinking-budget levels to fit the uplift-vs-`K` curve that S2T and §8 consume. This is the few-shot-onboarding requirement made concrete: probe fingerprints instead of weeks of live logs.
3. **Manufacture tiers**: AWQ-INT4/FP8 variants where self-hosted; each artifact re-validated on the golden set before joining a tier.
4. **Router update, no retraining**: Arch-Router in-context route + CEL tier addition, config-only; Stage-3 GraphRouter/UniRoute-class routers consume the probe fingerprint directly (few-shot node insertion).
5. **Shadow** (decision-only dry-run counterfactuals, zero spend) → **canary** (config-capped share, blue-green 2.1.2, judge-scored window, auto-rollback) → **full rotation**; priors shrink toward observed telemetry; circuit breaker guards the newcomer. Removal is the same path in reverse.

5.7 Dry-run & evaluation backbone (1.1.4)

Three modes over one artifact: (a) **decision-only dry-run** — counterfactual choices logged, zero spend; (b) **executed shadow** on s% traffic — true quality deltas at +s% cost; (c) **offline replay** over precomputed golden-set outcomes for every hosted model (RouterBench methodology) — free unlimited sweeps powering every `τ`, cascade gate, and cache threshold in this design. **promptfoo CI gate** (<5% golden-set regression) blocks every threshold/model/scorer change; per-model `τ` recalibration is a release gate on model swaps; judge-scored canaries with auto-rollback guard rollouts — silent quality regression is the #1 named risk and gets three independent tripwires (CI gate, canary, drift alarms).

---

6. Feature-flag & activation ladder

Every non-core optimization is a flag with an enable condition (measured) and an auto-degrade/kill condition. Enable-condition thresholds that rest on an [assumption]-tagged estimate in §8 are the measurement that upgrades the tag — the flag stays OFF until the measurement exists.

Flag	Component	Default / Stage	Enable when (measured)	Auto-degrade / kill when
`router.rules`	Config-rule routing	ON, MVP	always	never — the permanent floor
`router.arch`	Arch-Router-1.5B	ON, MVP	GPU slice available ∧ overhead p95 ≤ 75ms ∧ ≥ rules accuracy on golden set	overhead p95 > 2× target 15 min, or pod unhealthy → rules-only
`router.harden`	Audits + sanitization + cost-anomaly alarms	ON, MVP	always (audits are offline-free)	—
`guard.output`	Output guard (P3 step 3.9, 7.5.4)	ON, MVP (regex/dictionary layer); classifier layer Stage-2	regex layer: always — 7.5.4 mandates output-side guarding; classifier layer: co-located guard GPU available ∧ added p95 ≤ 15ms ∧ false-block rate < 0.5% on golden set	never fully off — classifier layer auto-degrades to regex-only on latency/health breach, response flagged `output_guard=degraded`
`router.condense`	Non-neural classifier condensation	OFF → Stage-2	p95 prompt length > `L_condense` on a real segment	any wiring of condensed text to safety guards (forbidden)
`predict.length`	DistilBERT length predictor	OFF → Stage-2	bucket accuracy ≥ 80% on logged traffic	rolling < 60% → static p90 priors
`cache.exact`	Tier-0 Redis	ON, MVP	always	Redis circuit open
`cache.semantic`	Raced semantic cache + vCache	OFF → Stage-2	hit rate ≥ 15% at observed error ≤ ε (re-baselined gate)	false-hit rate > ε for 24h
`cache.krites`	Async grey-zone promotion	OFF → Stage-3	curated tier ≥ 5k entries ∧ grey-zone ≥ 5% ∧ judge agreement ≥ 90%	judge agreement < 80%
`exec.cascade`	Sequential cascade	OFF → Stage-2	savings ≥ 40% at < 5% golden-set regression (brief's gate)	canary regression breach
`exec.thinkbudget`	Reasoning-tier thinking-budget dispatch (S2T)	OFF → Stage-2	reasoning tier provisioned (A15) ∧ golden-set uplift **per dollar-second** on the hard-`d` slice ≥ BoN's and ≥ refine's on the same slice (§8 test-time-reasoning table)	uplift/(dollar·s) falls below BoN/refine on recalibration, or reasoning-tier queue p95 breach (saturation)
`exec.speccascade`	Speculative parallel cascade	OFF → Stage-3	escalation rate ∈ [10%, 40%] ∧ escalated-tail deadline-miss > 5%	GPU util ≥ 85% sustained
`exec.bon`	Best-of-N / self-consistency	OFF → Stage-3	golden uplift ≥ +5pp on hard votable/verifiable slice ∧ GPU util < 70% ∧ uplift per dollar-second beats `exec.thinkbudget` on the same slice (else the built-in knob dominates — brief)	uplift < +2pp after recalibration
`exec.refine`	Verifier-gated refine loop	OFF → Stage-3	deterministic verifier exists for slice ∧ uplift ≥ +5pp on code/structured ∧ uplift per dollar-second beats `exec.thinkbudget` on that slice	—
`exec.hedge`	TTFT-based hedged dispatch	OFF → Stage-2	endpoint p95/p50 TTFT ratio > 3 ∨ time-budget-miss > 2%; cap +5% requests	measured overhead > 5%
`exec.moa`	MoA ensemble	OFF, Stage-3+	explicit max-quality product tier ∧ win-rate uplift ≥ +5pp vs best single at 5–10× cost (plausible mainly when closed APIs unreachable)	uplift below threshold

Flag	Component	Default / Stage	Enable when (measured)	Auto-degrade / kill when
`score.logprob`	Quantile logprob deferral	OFF → Stage-2	per-model calibration done	model swap without recalibration → hold at tier-0
`score.rm`	Reward-model scorer	OFF → Stage-3	sweep shows ≥ cascade-gate performance vs logprob-only	RM drift alarm
`score.judge_inpath`	In-path judge	OFF, Stage-3	high-budget slice with `T_rem ≥ 5·p95_judge` true > 1% of traffic	—
`serve.prefixlb`	Prefix-affinity LB (consistent hash)	OFF → Stage-2	> 1 replica per model	—
`serve.priority`	Deadline → vLLM priority	OFF → Stage-2	vLLM priority scheduling exposed by serving layer	—
`serve.specdecode`	Speculative-decoding latency tier	OFF → Stage-2	dedicated low-QPS latency tier provisioned	endpoint near saturation (speedup → ~1×)
`ctx.summarize`	Multi-turn history summarization	OFF → Stage-2	sessions > 10–20 turns exceed threshold share of traffic	coherence regression on golden multi-turn set
`compress.llmlingua2` / `compress.longllmlingua`	Neural prompt compression	OFF → Stage-3	real > 5k-token / RAG segment in telemetry ∧ ships **same release** as CompressionAttack hardening ∧ code slices no-compress	quality regression on affected slice
`router.trained`	RouteLLM-MF → quality-gap head → GraphRouter/UniRoute	OFF → Stage-3 (Phase-4 gate)	dry-run replay proves rules+cascade leave > 15–20% savings uncaptured	must beat RouterBench Zero-router baseline, else stays off

7. Technique placement map (complete inventory disposition)

Area	Technique	Disposition
Routing	Config-rule / heuristic tiered routing	**PLACED** — P1 step 1.7a, MVP; permanent fallback floor
Routing	Arch-Router-1.5B (archgw fork)	**PLACED** — P1 step 1.7b, MVP, `router.arch`, 75ms timeout; output doubles as XAI label
Routing	RouteLLM preference-trained routers	**PLACED** — Stage-3 (`router.trained`); MF variant first (near-free, µs-scale) when the >15–20% gate trips
Routing	Tri-metric prediction substrate (Hybrid-LLM)	**PLACED** — P1 step 1.8: cost+latency legs MVP (deterministic bookkeeping), learned quality-gap head Stage-3
Routing	GraphRouter / UniRoute few-shot cold-start	**PLACED** — pragmatic probe-fingerprint version in onboarding §5.6 (MVP); learned variant Stage-3 research axis
Routing	Router robustness hardening	**PLACED** — P1 step 1.10 inline + offline audits, MVP, `router.harden`
Routing	RouterBench offline simulation	**PLACED** — offline, MVP; powers dry-run replay + all threshold calibration (§5.7)
Cascading	FrugalGPT scorer-gated sequential cascade	**PLACED** — P2 strategy S3, Stage-2, `exec.cascade`
Cascading	Abstention-aware early-abstention trigger	**PLACED** — Stage-2, prompted (free) variant on the cheap tier; abstention short-circuits to DESC
Cascading	Unified cascade routing (Dekoninck)	**PLACED** — Stage-3 planner upgrade, gated on calibrated quality estimators + ≥3 tiers; edge collapses without them
Cascading	Deterministic tier-0 verifiers	**PLACED** — P3 step 3.1, MVP, mandatory first scorer layer

Area	Technique	Disposition
Cascading	Logprob quantile deferral (Chow rule)	**PLACED** — P3 step 3.2, Stage-2, `score.logprob`; per-model recalibration is a release gate
Cascading	DistilBERT-scale trained quality estimator	**PLACED** — Stage-3, replaces/augments logprob θ once 500–5k judge-labeled examples per task family exist
Cascading	Self-hosted LLM-as-judge	**PLACED** — async Win-Rate sampling MVP (P3 off-path); in-path only via gated 3.4, Stage-3, default OFF
Cascading	Threshold calibration + deadline-aware escalation guard	**PLACED** — guard = DESC predicate (MVP structure, Stage-2 active); calibration = §5.7 offline, MVP
Reasoning	Reasoning model as quality tier + thinking-budget knob	**PLACED** — P2 S2T, Stage-2, `exec.thinkbudget`; the **premium dial**: remaining T/C budget maps to a max-thinking-tokens cap debited on the ledger; self-hosted Qwen3-thinking / DeepSeek-R1-distill on vLLM; per the strategy brief it usually dominates hand-built best-of-N or refine for a small team, so `exec.bon` and `exec.refine` enable conditions are cross-gated against it (§6) and the comparison lives in §8
Caching	vLLM automatic prefix caching	**PLACED** — serving layer, MVP, default-on config; gateway's job is prefix discipline (step 1.5)
Caching	Prefix-cache-aware routing across replicas	**PLACED** — cheap consistent-hash affinity Stage-2 (`serve.prefixlb`); precise llm-d-style KV scheduling deferred to two-digit GPU counts
Caching	Exact-match response cache	**PLACED** — P1 step 1.3, MVP
Caching	Semantic cache (GPTCache/MeanCache class)	**PLACED** — P1 step 1.4 race + P2 S0 + P3 write, Stage-2, `cache.semantic`
Caching	vCache verified per-entry thresholds	**PLACED** — Stage-2, the default semantic-cache policy from day one of the cache
Caching	Krites async grey-zone promotion	**PLACED** — Stage-3 async worker, `cache.krites`
Caching	Cache poisoning defense + tenant isolation	**PLACED** — §5.4 write-gate + namespaces; design constraint from MVP, active with Stage-2 cache
Caching	Cache governance (TTL + hit accounting)	**PLACED** — MVP (accounting hook built before the semantic cache exists)
Compression	Static prompt adaptation (template/few-shot slimming)	**PLACED** — offline, MVP; promptfoo-verified trims
Compression	Prefix-cache-aware canonicalization	**PLACED** — P1 step 1.5, MVP
Compression	Non-neural router-path condensation	**PLACED** — P1 step 1.6, Stage-2, `router.condense`; never feeds safety classifiers
Compression	Multi-turn history summarization	**PLACED** — async between turns, Stage-2, `ctx.summarize`, coarse N-turn boundaries
Compression	LLMLingua-2 extractive compression	**PLACED** — Stage-3, `compress.llmlingua2`, gated on >5k-token telemetry; code = no-compress
Compression	LongLLMLingua (RAG, question-aware)	**PLACED** — Stage-3, only if RAG traffic materializes (3.3.1 hooks anticipate it)

Area	Technique	Disposition
Compression	Compression security hardening	**PLACED** — Stage-3, mandatory same-release pair with the first neural compressor
Compression	Abstractive trained compression (RECOMP/SCOPE/Cmprsr)	**EXCLUDED** — SFT+RL compressor training is the wrong effort profile for 6 people / 8 months / near-zero budget
Parallel	Best-of-N + self-consistency	**PLACED** — P2 S2 + P3 step 3.6, Stage-3, `exec.bon`; enable now cross-gated against `exec.thinkbudget` (§6)
Parallel	Verifier-gated best-of-N	**PLACED** — same slot; pick-rule = verifier pass-rate when `V=deterministic` (the "never pay for N>1 without a checkable signal" rule)
Parallel	MoA / LLM-Blender ensemble	**PLACED** — P2 S1, Stage-3+, `exec.moa` default OFF; the sanctions-era "beat closed APIs" option, only under an explicit max-quality tier
Parallel	Multi-model debate	**EXCLUDED** — dominated by self-consistency at equal compute; the gateway never selects it
Parallel	Hedged request dispatch	**PLACED** — P2 step 2.4, Stage-2, `exec.hedge`, TTFT-based, max 1, active loser abort
Parallel	Speculative cheap+strong parallel cascade	**PLACED** — P2 S4, Stage-3, `exec.speccascade`
Parallel	Racing semantic-cache lookup vs generation	**PLACED** — P1 step 1.4 / P2 S0, Stage-2; the only design compatible with 5.2 + 4.2.1
Parallel	Iterative refine loop (verifier + cap)	**PLACED** — P3 step 3.7 / DREF, Stage-3, `exec.refine`; never without a verifier; cross-gated against `exec.thinkbudget`
Scheduling	Budget ledger + deadline propagation + degradation ladder	**PLACED** — §5.1, MVP core; the spine
Scheduling	Output-length prediction	**PLACED** — P1 step 1.8: heuristic buckets MVP, DistilBERT Stage-2 (`predict.length`)
Scheduling	Retries + backoff/jitter + circuit breakers (+ cost-gated hedging)	**PLACED** — P2 steps 2.4–2.5, MVP (hedge Stage-2); existential for the closed-API tier
Scheduling	Health/load/prefix-aware LB + rate-limit dispatch + KubeAI autoscaling	**PLACED** — health-aware MVP; prefix-affinity Stage-2; rate-limit windows MVP async; autoscaling = serving-layer config, MVP
Scheduling	Deadline→vLLM priority mapping (QLM-lite)	**PLACED** — P2 step 2.3, Stage-2, `serve.priority`
Scheduling	Full QLM global queue manager	**EXCLUDED** (revisit-gated) — adopt only if priority mapping measurably leaves SLO attainment on the table at scale
Scheduling	Speculative decoding endpoints	**PLACED** — serving-layer config, Stage-2, `serve.specdecode`; gateway only routes tight-`T` requests there
Scheduling	Disaggregated prefill/decode (DistServe/Splitwise/llm-d)	**EXCLUDED** — premature below two-digit GPU counts; endpoint abstraction kept P/D-agnostic so later adoption costs zero gateway changes
Scheduling	Quantization tiers (AWQ-INT4 / FP8)	**PLACED** — offline, MVP; manufactures the price-quality gradient the router monetizes; per-artifact promptfoo re-validation
Quality	Self-hosted LLM judge (async)	**PLACED** — async worker A1, MVP

Area	Technique	Disposition
Quality	Small RM scorer (Skywork 0.6–8B)	**PLACED** — P3 step 3.3, Stage-3, `score.rm`
Quality	Output-side guard (leakage / policy scan, 7.5.4)	**PLACED** — P3 step 3.9, MVP regex layer + Stage-2 classifier layer, `guard.output`; distinct `output-policy-violation` error class; streamed via rolling-window holdback
Quality	promptfoo golden-set harness	**PLACED** — offline CI, MVP day one (200–500 requests, growing to 1000+)
Quality	Online telemetry + sampled Win Rate	**PLACED** — P3 step 3.13 + async A1, MVP (hard metrics on 100% of requests)
Quality	Dry-run counterfactual + shadow/replay	**PLACED** — DDRY branch + §5.7, MVP
Quality	CI quality gate + judge-scored canary + auto-rollback	**PLACED** — §5.7 + async A4, MVP
Quality	Template-first XAI	**PLACED** — P3 step 3.11 / §5.5, MVP
Quality	End-user feedback capture	**PLACED** — Stage-2, out-of-band endpoint keyed on request id; calibration signal only, never primary telemetry

---

8. Comparative technique analysis — impact estimates with epistemic labels

This section is deliverable (a) in full: per-area comparative tables with expected quality / cost / latency deltas **for this workload** (mostly-short, easy-skewed, Persian/English mixed chat on single-digit H100s — A1, A5, A9). Every estimate carries one of three tags:

- **[evidence]** — a published paper, benchmark, or measured number transfers with its stated conditions;
- **[reasoned]** — derived from this project's constraints by arithmetic or dominance argument, no external measurement;
- **[assumption]** — needs measurement on our traffic before it can be trusted; the matching §6 enable condition **is** that measurement, and the flag stays OFF until it exists.

Where a benchmark number is cited but its transfer to this workload is unproven, the cell carries both tags. Numbers quoted elsewhere in this document (45–85% routing savings, 2–5% hedge cost, 51±12ms router latency, the ">30% hit rate" gate) inherit the labels assigned here.

8.1 Routing

Technique	Quality Δ	Cost Δ	Latency Δ	Basis
Rules + Arch-Router policy routing	−0–2pp vs always-strong at the routed operating point	−45–85% on benchmark mixes [evidence: RouteLLM reports ~85% cost reduction at 95% GPT-4 quality on MT-Bench; RouterBench mid-range 40–60%]; on our easy-skewed Persian/English mix [assumption → §5.7 dry-run replay before any savings claim]	+51±12ms [evidence: Arch-Router paper, their hardware]; on our GPU slice [assumption → `ROUTER_TIMEOUT` p95 measurement, §6 `router.arch` gate]	mixed
Rules-only floor	quality = table lookup, no model risk	savings smaller than learned routing by 10–20pp [reasoned: coarser partitions]	<1ms [evidence: CEL eval benchmarks]	mixed
RouteLLM-MF trained router	≈ equal quality	additional −10–20% vs heuristic routing [evidence: RouteLLM ablations]; on our logged traffic [assumption → `router.trained` gate: >15–20% uncaptured savings in replay]	~µs [evidence: MF is a dot product]	mixed

Technique	Quality Δ	Cost Δ	Latency Δ	Basis
Tri-metric cost/latency legs	—	prediction error = length-predictor error ±20–40% [reasoned: bucketed-length arithmetic; absorbed by margins `m_t`, `m_c`]	<1ms [reasoned: table lookups]	reasoned
Router hardening (OOD, gadget-suffix, anomaly counters)	prevents adversarial forced-escalation cost blowups [reasoned: attack surface exists whenever routing is model-based]; incidence on our tenants [assumption → cost-anomaly telemetry]	—	<1ms	reasoned/assumption

8.2 Cascading & scoring

Technique	Quality Δ	Cost Δ	Latency Δ	Basis
Sequential cascade (FrugalGPT class)	≤5% regression targeted	up to −98% on benchmark suites [evidence: FrugalGPT]; realistic gate here ≥ −40% [reasoned: brief's gate, discounting benchmark-specific tuning]; actual escalation rate on our mix [assumption → `exec.cascade` enable]	+1 cheap-tier generation on escalated requests [reasoned: arithmetic]	mixed
Logprob quantile deferral	deferral quality beats raw-mean confidence [evidence: quantile-based deferral literature, Gupta et al. class results]	scorer cost ≈ 0 [evidence: logprobs free on vLLM]	~0ms	evidence; per-model `τ_lp` transfer [assumption → release-gate recalibration]
Tier-0 deterministic verifiers	hard floor on structured/code slices; eliminates a class of silent failures [reasoned]	≈ 0	0–5ms (code exec bounded 100ms–2s)	reasoned
RM scorer (0.6–8B)	+2–8pp selection uplift on open-ended [evidence: RewardBench-class correlation results; magnitude on Persian open-ended [assumption → A8 labels + `score.rm` sweep]]	one prefill pass [reasoned]	+10–50ms [reasoned: model size × prefill length]	mixed
In-path judge	highest-fidelity single scorer [evidence: MT-Bench ~80% judge–human agreement]	+1 judge generation	+300ms–3s [reasoned] — hence gated to `T_rem ≥ 5·p95_judge`	mixed

8.3 Caching

Technique	Quality Δ	Cost Δ	Latency Δ	Basis
Exact cache	0 (byte-identical answers)	−100% marginal on hits; hit rate on our traffic [assumption → logged telemetry; no literature anchor is trusted]	−(full generation) on hits, +<1ms on misses [reasoned]	reasoned/assumption
Semantic cache + vCache	false-hit rate bounded ≤ ε = 1% [evidence: vCache guarantee]	−(hit rate × full generation); the 31% hit-rate anchor is a 20-user study [evidence, weak → assumption for our market; §6 gate re-baselined to ≥15% measured]	0ms added on miss (raced) [reasoned: race design]; −latency on hits	mixed

Technique	Quality Δ	Cost Δ	Latency Δ	Basis
Tenant isolation + verified-write gate	prevents cache poisoning [evidence: CacheAttack, 80–88% hijack success on shared production caches]	negligible overhead [reasoned]	—	evidence
Krites grey-zone promotion	grows curated hit tier without inline risk [reasoned]; grey-zone share ≥5% on our traffic [assumption → §6 gate]	async judge cost, off-peak [reasoned]	0ms critical-path [reasoned]	reasoned/assumption
vLLM prefix caching + canonicalization	0	−(shared-prefix prefill)	TTFT −30–90% on shared-prefix traffic [evidence: vLLM APC benchmarks; magnitude workload-dependent → assumption on our template share]	mixed

8.4 Compression & context

Technique	Quality Δ	Cost Δ	Latency Δ	Basis
Non-neural router-path condensation	intent-classification quality preserved [reasoned: classifier sees gist]; **jailbreak detection −14.2pp if guards see condensed text** [evidence: fact-checked verdict from the inventory — the one labeled area; this is why step 1.6 structurally forbids feeding guards condensed text]	0 (router input only)	~19ms CPU [evidence: cited measurement], only > `L_condense`	evidence
LLMLingua-2 extractive compression	2–5× compression at small loss on English benchmarks [evidence: LLMLingua-2 paper]; code slices degrade [evidence] → no-compress rule; Persian behavior [assumption → A9 acceptance test before Stage-3 enable]	−input tokens proportional	+compressor inference [reasoned]	mixed
Multi-turn history summarization	coherence risk on long sessions [assumption → golden multi-turn set, §6 kill condition]	−50–80% history tokens on >10-turn sessions [reasoned: arithmetic on turn lengths]	0ms inline (async between turns) [reasoned]	reasoned/assumption
Static prompt slimming	0 (promptfoo-verified per trim) [reasoned]	−5–15% input tokens [reasoned: template audit estimate]	—	reasoned

8.5 Parallel & ensemble

Technique	Quality Δ	Cost Δ	Latency Δ	Basis
Best-of-N / self-consistency	+5–18pp on votable/verifiable benchmarks (GSM8K-class) [evidence: self-consistency literature]; uplift on our hard-`d` slice [assumption → `exec.bon` +5pp gate]	×N generation minus shared prefill [reasoned: arithmetic]	≈ single-shot wall-clock when batched [evidence: vLLM `n>1`]	mixed
Verifier-gated BoN	dominates plain BoN where `V=deterministic` [reasoned: selection signal is exact]	same ×N	same	reasoned
MoA ensemble	beats single frontier models on AlpacaEval 2.0 [evidence: MoA paper]; on our open-ended Persian slice [assumption]	×5–10 [reasoned: member count × aggregator]	×2–2.5 [reasoned: two rounds]	mixed
Multi-model debate	no gain over self-consistency at equal compute [evidence: Smit et al.] → **excluded**	—	—	evidence

Technique	Quality Δ	Cost Δ	Latency Δ	Basis
Hedged dispatch	0 (same answer, earlier)	+2–5% [evidence: tail-at-scale hedging literature, non-LLM systems; transfer to vLLM TTFT tails [assumption → `exec.hedge` gate: p95/p50 ratio > 3 measured]]	tail p99 cut 20–70% in the source domain [evidence, same caveat]	mixed
Speculative parallel cascade	= cascade quality	+ĉ(cheap) wasted when strong wins [reasoned: arithmetic]	= strong-only latency, removing cascade's serial penalty [reasoned]	reasoned; GPU headroom [assumption → util telemetry]

8.6 Test-time reasoning (the premium dial)

Technique	Quality Δ	Cost Δ	Latency Δ	Basis
Reasoning-tier thinking-budget knob (S2T, `exec.thinkbudget`)	roughly log-linear quality gain in thinking tokens on math/code/hard-reasoning [evidence: o1/R1/Qwen3 test-time-compute scaling reports]; **usually dominates hand-built BoN/refine at equal spend for a small team** [reasoned: strategy brief's stated position — one knob, no orchestration code, no selector to build]; uplift per dollar-second vs BoN on our golden hard-`d` slice [assumption → the §6 `exec.thinkbudget` enable condition is exactly this measurement]	+thinking tokens at tier price, capped by `K` derived from `C_rem` [reasoned: ledger arithmetic]	+thinking time, capped by `K` derived from `T_rem` [reasoned]; watchdog-truncatable [reasoned: §3]	mixed
Refine loop (verifier-fed)	+5–15pp on code/structured with a real verifier [evidence: self-debug/self-refine literature]; ≈0 or negative without one [evidence: same literature] → DREF's "never without a verifier" rule	+1 generation/turn	+1 generation/turn serial	evidence; our slice sizes [assumption → `exec.refine` gate]
BoN vs knob vs refine (the three-way choice)	resolved empirically per slice: §6 cross-gates `exec.bon` and `exec.refine` on beating `exec.thinkbudget` per dollar-second; default preference order S2T → BoN → refine reflects the brief	—	—	reasoned (ordering) / assumption (winner per slice)

8.7 Scheduling & serving

Technique	Quality Δ	Cost Δ	Latency Δ	Basis
Quantization tiers (AWQ-INT4/FP8)	≤1–2pp loss on common benchmarks [evidence: AWQ/FP8 eval literature]; per-artifact loss on our golden set [assumption → mandatory promptfoo re-validation before tier admission]	−50–70% serving cost per token [evidence: throughput gains at same hardware]	TPOT improves [evidence]	mixed
Deadline→priority mapping	0	0	deadline-miss reduction under load [reasoned: queueing argument; magnitude [assumption → measure before considering full QLM]]	reasoned/assumption
Speculative-decoding tier	0 (lossless) [evidence]	+draft-model overhead	×1.5–2.5 TPOT at low QPS, →×1 at saturation [evidence: speculative-decoding literature] → §6 kill condition	evidence
Length predictor	—	tightens cost predictions from ± (20–40)% heuristic error toward bucketed ±15% [assumption → `predict.length` 80% bucket-accuracy gate]	+4ms	assumption

Technique	Quality Δ	Cost Δ	Latency Δ	Basis
Retries/breakers/hedges as a family	availability floor under closed-API loss [reasoned: sanctions scenario is a stated operating condition]	bounded by ledger debits [reasoned]	bounded by DR predicate [reasoned]	reasoned

8.8 Quality measurement & guards

Technique	Quality Δ	Cost Δ	Latency Δ	Basis
Async judge Win-Rate sampling	metric fidelity ~80% agreement with humans on English [evidence: MT-Bench]; Persian agreement unvalidated [assumption → A8/A9 acceptance test]	1–2% of traffic × judge cost, off-peak [reasoned]	0ms inline	mixed
Input guards (PII + injection)	detection quality degrades −14.2pp on condensed text [evidence — hence full-text-only rule]; Persian PII pattern recall [assumption → A9]	≈0	1–5ms	mixed
Output guard (P3 3.9)	regex layer: high precision on format-detectable secrets/raw PII — raw PII in output is leakage by construction given input redaction [reasoned: structural argument]; classifier recall on Persian leakage/harm categories [assumption → measured false-block <0.5% and recall audit before the Stage-2 classifier layer promotes, §6 `guard.output`]	≈0	+1–5ms regex, +5–15ms classifier; holdback buffer adds bounded streaming delay [reasoned: buffer arithmetic inside `t_floor`]	reasoned/assumption
CI gate + canary + drift alarms	bounds silent regression to <5% golden-set delta per change [reasoned: gate definition]	offline only	0ms	reasoned

Cross-reference rule. Every [assumption] tag above maps to a named §6 enable condition or an §9 resolving question; no flag whose expected benefit rests on an [assumption] ships ON. The three tags are maintained in the decision-record schema so dry-run replay (§5.7) can upgrade [assumption] → [evidence] mechanically as logged traffic accumulates.

---

9. Assumption register

Every assumption forced by a gap in the project docs, with the question that resolves it.

#	Assumption made	Resolving question
A1	Launch traffic is mostly short (<2k tokens), single-turn, easy-skewed chat; all length/difficulty gates (`L_condense`, compression deferral, cascade savings targets) assume this	Can Arvan provide a sample or characterization of expected tenant workloads (length distribution, task mix, Persian/English ratio, multi-turn share)?
A2	The cost budget `C` is denominated in a normalized per-token price the operator assigns to self-hosted models (GPU-second-derived), same unit as closed-API prices	What is the official cost basis and currency of `C`, and who sets price/token for self-hosted models?
A3	The time budget `T` bounds total wall-clock to the **final** token; for streamed responses the watchdog truncates generation at the deadline rather than aborting the stream	Does `T` bound completion or first token, and what is the contract for streamed responses that hit the deadline mid-stream?
A4	The optional quality target `Q` is a policy-relative score in [0,1], mapped internally to golden-set percentiles and the semantic-cache error bound ε	What scale/semantics does the quality target use, and how is "target met" defined to the client?
A5	MVP fleet is single-digit H100s with ~1 replica per model — hence prefix-affinity LB, precise KV scheduling, and P/D disaggregation are deferred	What are the launch GPU fleet size, replica counts, and 12-month growth expectation?
A6	Multi-tenant public deployment from day one (drives per-tenant cache namespaces, RBAC, rate anomaly alarms at MVP)	Is the MVP a single internal tenant or public multi-tenant, and when does the second paying tenant arrive?
A7	Closed-API dispatch is disabled by default per tenant; when enabled, only redacted content may leave the infrastructure	Is off-infrastructure dispatch ever permissible, and under what data-residency rules (7.9.1) per tenant class?

#	Assumption made	Resolving question
A8	No human-annotation budget exists; golden-set quality labels are team-authored and judge agreement on Persian content is unvalidated	Is there any budget for human labeling, and do we have (or can we build) a Persian-language eval set to validate the self-hosted judge and PII patterns?
A9	Arch-Router, MiniLM-class embedders, and regex PII patterns perform acceptably on Persian text — none of the cited benchmarks tested Persian	What fraction of traffic is Persian, and what is the acceptance test for Persian routing/embedding/PII accuracy before MVP launch?
A10	The serving layer (company-owned per 6.2) exposes request cancellation and `--scheduling-policy priority` to the gateway; cancellation discipline (2.6) and `serve.priority` depend on it	Does the company's serving layer grant the gateway abort and priority APIs on vLLM endpoints, and who owns that contract?
A11	No tenant-level budget override exists at MVP; 7.3.1's "explicit configured override" is a Stage-2+ admin feature	Is a budget-override knob required at launch, and who is authorized to set it?
A12	The user priority-profile "panel" (4.1.2) is satisfied at MVP by per-tenant config + request headers; a UI panel is out of gateway-core scope for the 8 months	Is a self-service UI panel a deliverable within the 8-month roadmap, or is API/config-level profile selection sufficient?
A13	`t_floor` = max(200ms, 2%·T) suffices for Phase-3 + assembly at p95, **including the output guard and its streaming holdback**; unmeasured until a running system exists	Measure Phase-3 p95 in the first dry-run/shadow window — what tail reserve does real traffic need?
A14	Sessions (1.1.3) are implemented as gateway-side history storage in Redis keyed by session id, replayed into stateless requests	Is session state gateway-owned or client-supplied per request, and what are its retention rules (3.2.3)?
A15	A self-hosted reasoning model (Qwen3-class thinking mode or DeepSeek-R1 distill on vLLM) fits the launch fleet alongside the standard tiers, possibly sharing hardware with the strong tier; S2T and the max-thinking-tokens sampling param depend on it	Can the launch fleet host a reasoning tier (dedicated or shared), and does the serving layer expose a thinking-token cap per request?

10. Requirement traceability (spot map)

Requirement	Where satisfied
4.1.1 tri-metric prediction	P1 step 1.8 (cost/latency deterministic MVP; quality priors from probes; learned head Stage-3 gated)
4.1.2 priority profiles	P1 step 1.1 merge → sort inversion in 1.9
4.1.3 / 5.5 meta-learning onboarding, hot-swap	§5.6 probe-fingerprint pipeline, config-only routes, Stage-3 learned variant
4.1.4 XAI	§5.5 template-first, faithful by construction
4.1.5 Win Rate / Cost Savings / TTFT / Router Overhead	P3 step 3.13 + async A1; `router_overhead_ms` targets §5.2
4.2.1 / 6.3.6 ms-scale overhead	§5.2 allowlist + auto-degrade
6.4.1–6.4.4 / 7.1.1 / 7.3.1 budget contract	§5.1 ledger + watchdog + pre-spend reservations; DESC/DREF guards
6.4.5–6.4.8 optional strategies + target-not-met	P2 S0–S5 incl. S2T, P3 DQ
6.5.2 multi-candidate selection	P3 step 3.6
6.5.3 per-call timeout	P2 step 2.2
6.5.5 / 7.4.x retry, breakers, graceful degradation	P2 steps 2.4–2.5, failure tables, degradation ladder
1.1.4 dry-run	DDRY branch + §5.7
6.6.1–6.6.4 caching, accounting, per-tenant	§5.4
1.1.2 / 4.2.4 / 3.2.2 / 7.5.4 PII + input **and output** guards	§5.3 (both directions), P1 step 1.2 (input), P3 step 3.9 + DOG (output, per-tenant configurable, distinct error class)
1.1.3 / 5.1 sessions on a stateless core	ledger in request context; session state in Redis (A14)
5.3 non-blocking	raced cache, async workers, streaming dispatch

Requirement	Where satisfied
2.1.2 / 2.2.x blue-green, dynamic config, validation	§5.6 steps 1 & 5, §5.7 canary
7.10.2 distinct error classes	§5.1 four-outcome rule; D0/D5/DOG
6.8.x / 7.7.x declarative config, no client change	§0 tunables, §5.6, flag ladder §6
Deliverable (a) comparative technique analysis with labeled impact estimates	§8 per-area tables, [evidence]/[reasoned]/[assumption] tags cross-referenced to §6 enable conditions

---

The one-sentence architecture: rules plus a 1.5B policy router pick the cheapest feasible model in under 65ms; a ledger that every call debits before dispatch and a watchdog that fires at `deadline − t_floor` make budget violations structurally impossible; free signals (verifiers, logprobs, caches) decide when to spend more — and when they do, leftover budget buys thinking tokens on a reasoning tier before it buys hand-built orchestration; every answer passes an output guard before it leaves; and every clever component from the research ceiling sits behind a feature flag whose enable condition is a measurement, not an opinion.